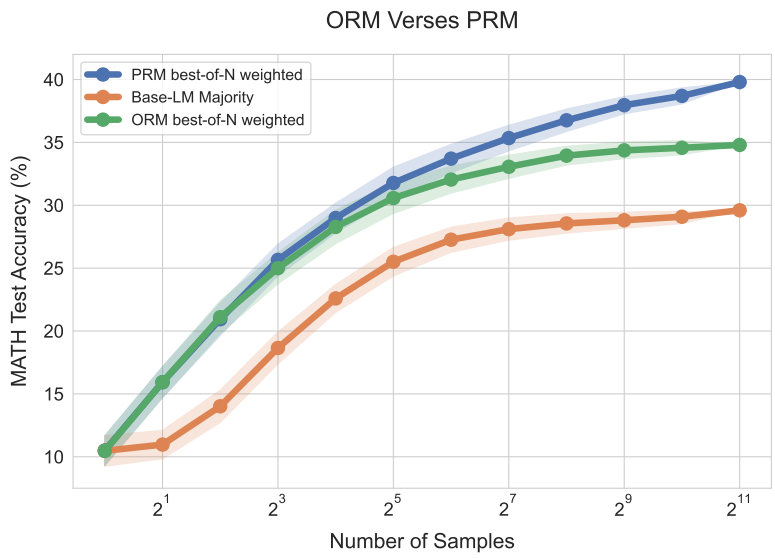

Figure 14 | We compare PRM and ORM models finetuned from PaLM 2-S* in a best-of-N evaluation. We use the PaLM 2-S* base LM to sample outputs, using a few-shot prompt. We see that the PRM greatly outperforms the ORM at a larg number of samples.

of samples used. We use the last step prediction from the PRM to score the answers as described in Appendix E.

G. Prompting Details

In order to enable the base model to output answers in a step-by-step format to which a PRM can be applied, we use a 4-shot prompt consisting of randomly selected correct answer examples from the PRM800k data released by Lightman et al. [22]. Specifically we use answers from the phase 1 training split. These answers correspond to GPT-4 generated correct answer examples, which include the correct step-by-step format. In initial experiments, we found that this prompting procedure produces similar results to the prompt used in Lewkowycz et al. [20]. We use this prompt for generating training data for the PRM and the revision model. We also use this prompt when conducting search against the PRM on the test-set. To grade the final answer predicted by this prompt, we use the grading function released by Lightman et al. [22].

H. Revision Model Finetuning Details

For fine-tuning the revision model, we follow the procedure outlined in Section 6.1. We first sample 64 outputs per question. We then filter out all answers which end in an invalid solution. For each correct answer, we then sample a number uniformly between 0 and 4 indicating how many incorrect answers to include in context for training. The correct answer is used as the last answer in the trajectory (which we train the model to produce) and the incorrect answers are included in context. If the sampled number is greater than 0, we then find the closest incorrect answer according to a character-level edit distance metric to include as the last incorrect answer in the trajectory. The goal here is to select an incorrect

answer which is somewhat correlated with the correct answer, to improve learning. The remaining incorrect answers, we sample randomly from the set of available answers. In the case where there are fewer than 4 incorrect answers sampled, we truncate the uniform distribution's max to match the number of incorrect samples. We use this procedure to generate trajectories for all questions in the training data.

We then finetune the base language model on the correct answer solutions in these generated trajectories. We use the AdamW optimizer with lr 1e-5, batch size 128, dropout 0.0, and Adam betas $(0.9, 0.95)$.

We find that generally evaluating loss on an evaluation set consisting of trajectories generated as described above, does not provide a good signal for early stopping. Rather, we find that checkpoints much after the evaluation loss begins increasing are much more capable of revisions. This is likely because after finetuning the revision model, the evaluation set represents off-policy data, which will naturally be out-of-distribution compared to the trajectires that the model itself would generate on-policy. We therefore select our revision model checkpoint slightly after the point where we observe overfitting on the validation set.

I. Revision Model Selection Criteria

As described in Section 6.1, in order to effective use our revision model we need to deploy a criteria for selecting the best answer both within a revision trajectory and between multiple parallel trajectories. We use two approaches: 1) ORM verifier; and 2) majority voting.

For the ORM verifier, we train an ORM on the revision model's outputs according to the procedure in Appendix J. At inference, time we then use this verifier to select the best answer. Since we have two axes across which to aggregate (within each revision trajectories and between multiple trajectories), we deploy a hierarchical strategy, first selecting the best answer within each revision trajectory and then aggregating these selected answers across trajectories. To select the best answer within each trajectory, we perform best-of-N weighted aggregation and then choose the highest scoring solution with the maximum best-of-N weighted answer. Then, to select the final answer across all revision chains, we perform another round of best-of-N weighted selection using the best answer from each revision chain. The answer after this second round of best-of-N weighted represents our final answer prediction.

For majority voting we found hierarchical aggregation to create problems when the length of the trajectory or the number of trajectories was too small. The problem being that without enough samples, majority voting is unable to effectively select the best option. Therefore, for majority voting, we simply take all answers, across all trajectories, at once and take their majority as the final-answer. We found this to produce much smoother scaling behavior than the hierarchical approach.

J. Revision Model Verifier Training

We found that the PRM we finetuned on the PaLM 2-S* base model outputs was not as effective when applied to the PaLM 2-S* revision model's outputs (see Figure 15(a)), likely due to distribution shift with the revision model. We therefore, trained a separate ORM verifier to use with our PaLM 2-S* revision model. We could have trained a PRM as well, but opted for an ORM due to the high cost of generating per-step PRM labels.

We modified the standard ORM slightly for the revision setting, by finetuning the ORM with previous revision in context, such that the verifier has access to the same context as the revision model, allowing

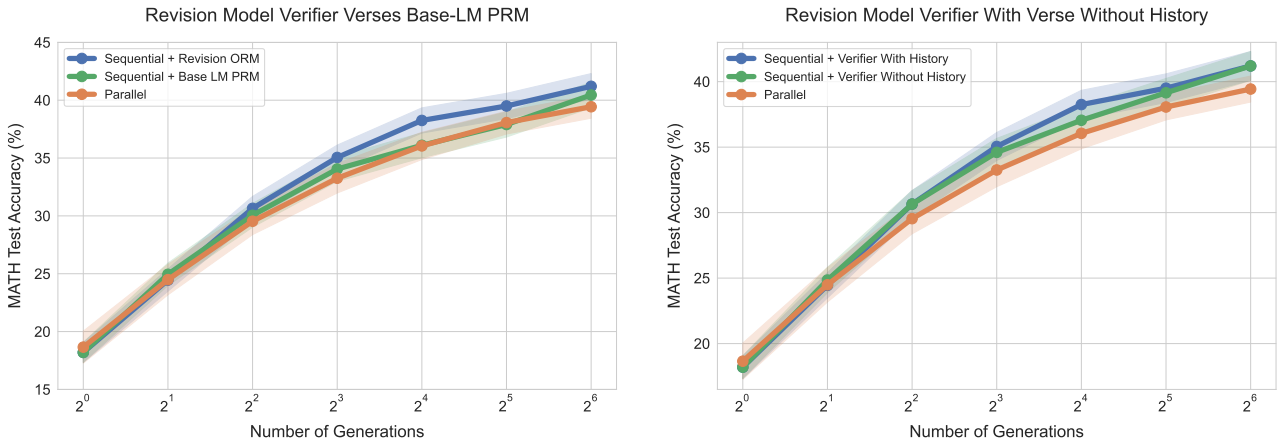

Figure 15 | Left: we compare the ORM we trained on the revision model's outputs against the PRM we trained on the PaLM 2-S* base model's outputs. We see that when applied to outputs from the revision model, the ORM adapted to the revision model outperforms the PRM, likely due to distribution shift with the revision model. Right: we ablate the effect of including previous revisions in the revision model verifier's context. We see that including revisions in-context helps the verifier slightly, but both settings still outperform the parallel baseline.

the verifier see the revision model's previous answer attempts when scoring the current answer. All other experiment details are identical to those used for training the PRM.

Empirically, we find that including the revision history in context improves performance slightly (see Figure 15(b)). Additionally, even without the revisions in context, we see that sequential revisions still slightly outperforms parallel, demonstrating improvements from sequential sampling are not just due to the verifier's context.

K. ReST^{EM} Revision Model Experiments

We experimented with further optimizing our PaLM 2-S* revision model by training the model with a simplified RL algorithm: ReST^{EM} [35]. Specifically, we generated 64 revision trajectories of maximum length 5 for each question on the MATH training set. We stopped the revision model at the first correct answer in each trajectory. Using this generated data, we then finetuned the base LM on the correct answer data. To help the model learn the task, we explicitly balanced the distribution of trajectory lengths.

In Figure 16, we plot the performance of this new revision model as we vary the sequential to parallel ratio. We see that additional sequential revisions substantially hurts performance with this new model. We hypothesize that this degradation is due to the fact that the online data obtained from running ReST^{EM} exacerbates spurious correlations in revision data, causing the optimized model to fail to learn the revision task. We believe that using a more offline data collection strategy, as done in Qu et al. [28], may be more effective, and leave further exploration to future work.